explain me the entire report in simple for my hackathon prototype buiding

means explain me eeach concepts mentioned in this along with the step by step process to build the best prototype fro this
tell me the entire process to build the desired platfrom for this along with step by step pipeline to build it
explain small to small concpts related to both technical and the problem related terms

This report proposes PARAKH, an AI-assisted tender evaluation platform for CRPF, but the real idea is not "AI reads PDFs." It is "AI helps officers evaluate vendors in a way that is explainable, auditable, and safe under government scrutiny." [1]
For your hackathon prototype, the best approach is to build a smaller but very convincing version of this platform: upload a tender, extract criteria, upload bidder documents, map evidence to criteria, show Eligible / Not Eligible / Manual Review, and generate an explanation with audit history.[1]

Core idea

The report says the real procurement problem is not just slow document reading; it is the risk officers face when a decision cannot be explained later to auditors, RTI applicants, vigilance teams, or courts.[1]
So your platform should not behave like a black-box AI judge; it should behave like an assistant that shows clause, evidence, extracted value, confidence, and reason behind every suggestion.[1]

In simple words, think of the system like this:

- Tender document tells the rules.[1]
- Bidder documents provide proof.[1]
- Rule engine compares proof with rules.[1]
- Ambiguous cases go to a human reviewer, not auto-rejection.[1]
- Every action is saved as an audit trail.[1]

That is the heart of the whole report: officer remains the final decision-maker, and the system exists to improve consistency and defensibility.[1]

Problem terms

Here are the small concepts in very simple language:

- **Tender:** A government request asking vendors to supply goods or services.[1]
- **Bidder:** A company applying for that tender.[1]
- **Eligibility analysis:** Checking whether the bidder satisfies all required conditions.[1]
- **Criterion:** One rule to check, such as turnover, ISO certificate, experience, or local content percentage.[1]
- **Corrigendum:** An official correction or update to the original tender.[1]
- **BoQ:** Bill of Quantities, usually a table showing item-wise cost or quantity details.[1]
- **RTI:** Right to Information; people can later ask why a bidder was accepted or rejected.[1]
- **CAG/CVC/Vigilance:** Oversight and audit bodies that may inspect the procurement decision.[1]
- **Defensibility:** Ability to justify the decision with proof and logic.[1]
- **Explanation gap:** When the system gives a result but cannot clearly show how it reached it.[1]

The report gives many examples of why two officers may reach different conclusions, such as interpreting “similar works,” turnover periods, handwritten certificates, GSTIN typos, faded certificates, or local-content mismatch differently.[1]
That means your prototype must show that it handles ambiguity carefully instead of pretending every answer is certain.[1]

Full architecture

The report divides the platform into four major subsystems: Tender Understanding Engine (TUE), Bidder Document Intelligence (BDI), Evaluation and Ambiguity-Resolution Engine (EARE), and Human-in-the-Loop Workbench with Audit System (HLWAS).[1]
In simple terms: TUE reads the tender rules, BDI reads bidder documents, EARE compares both and gives a verdict, and HLWAS lets an officer review and export the final record.[1]

1. TUE

TUE reads tender-side documents like NIT, qualifying requirements, annexures, forms, corrigenda, addenda, and pre-bid minutes, then creates a structured “criteria manifest.”[1]
This manifest is the blueprint used later during evaluation, and it stores each criterion with fields like criterion_id, full text, type, threshold, evidence template, and validation rules.[1]

Important TUE concepts:

- **Clause detection:** Breaking tender text into meaningful rule statements.[1]
- **Classification:** Labeling a clause as financial, experience, technical, certification, compliance, or non-criterion.[1]
- **Hard criterion:** A strict rule, like turnover must be at least 50 Cr.[1]

- **Graduated criterion:** A subjective rule, like “similar works” or “adequate experience,” which usually needs human review.[1]
- **Normalization:** Converting dates, currency, and units into standard format.[1]
- **Versioning**: Keeping updated manifests whenever corrigenda change the tender.[1]

For the prototype, you do not need full Indic-BERT training; you can manually define 5 to 10 criteria or use regex and keyword rules to extract them.[1]

2. BDI

BDI handles bidder documents, which may be digital PDFs, scans, handwriting, mobile photos, tables, bilingual text, or faded papers.[1]
Its role is to convert messy real-world documents into structured fields like turnover value, ISO certificate number, GSTIN, local content percentage, and years of experience.[1]

Important BDI concepts:

- **OCR**: Optical Character Recognition, converting image text into machine-readable text.[1]
- **Pre-processing**: Improving image quality before OCR using contrast enhancement or dewarping.[1]
- **Routing**: Detecting document type and sending it to the right extraction pipeline.[1]
- **Field extraction:** Pulling required values from a page or table.[1]
- **Confidence score**: A number showing how reliable the extracted field is.[1]
- **Entity resolution:** Matching names even if they are written differently, often anchored using PAN or GSTIN.[1]
- **Red-flagging**: Detecting suspicious or poor-quality documents, but not auto-rejecting them.[1]

For your prototype, use:

- PyMuPDF or pdfplumber for digital PDFs.[1]
- PaddleOCR or Tesseract for scanned pages.[1]
- Simple image preprocessing with OpenCV.[1]
- Regex extraction for PAN, GSTIN, dates, turnover, certificate validity, and percentages.[1]

3. EARE

EARE is the logic engine that maps evidence to criteria and produces a verdict.[1]
The report uses a deterministic three-state model: Eligible, Not Eligible, or Needs Manual Review.[1]

Simple flow:

1. Take one criterion from the manifest.[1]
2. Find relevant evidence from bidder documents.[1]

3. Extract and normalize the value.[1]
4. Apply a fixed rule.[1]
5. If clear pass, mark Eligible.[1]
6. If clear fail, mark Not Eligible.[1]
7. If low confidence, missing evidence, or ambiguity, mark Needs Manual Review.[1]

This is one of the strongest parts of the report because it avoids risky auto-decisions on unclear documents.[1]

4. HLWAS

This is the officer-facing interface.[1]
The report describes a tri-pane workbench with criterion context on top, evidence viewer in the middle, and verdict plus reason capture at the bottom.[1]

Important concepts:

- **Override**: Officer changes the suggested verdict.[1]
- **Reason code**: Structured reason for override, like tolerance applied or similar work precedent.[1]
- **Clarification request**: Asking the bidder to upload a better or missing document.[1]
- **Committee mode**: Multiple officers collaborate and vote on technical review.[1]
- **Audit export**: Final Excel/PDF/ZIP output for reporting and legal traceability.[1]

For the hackathon, a simpler dashboard is enough: one screen showing criterion list, verdict status, evidence page preview, extracted value, confidence, and editable reviewer note.[1]

Best prototype scope

To build the **best** hackathon prototype, do not attempt all 29 pages of the report as full production features. The report itself proposes a phased plan and even marks some features as deprioritized for the proof of concept, such as full committee real-time mode, blacklist API integration, automated retraining, and a mobile app.[1]
So your winning prototype should focus on the strongest, most demonstrable chain: tender upload, criteria manifest, bidder upload, evidence extraction, deterministic verdict, manual review, and audit report.[1]

Build around one tender use case with 3 to 5 bidder packs and 5 to 8 criteria such as:

Criterion	Prototype rule
Average turnover	Must be >= threshold from tender. [1]
ISO/BIS validity	Certificate must exist and be valid on bid date. [1]
Experience	Must show similar work or push to manual review if wording is subjective. [1]
GSTIN/PAN	Must be extracted and validated; mismatch goes to review. [1]

Criterion	Prototype rule
Indigenous content	Compute from BoQ and compare with declaration. [1]
Missing documents	Mark manual review or not eligible based on policy. [1]

This scope is realistic and still captures the full spirit of the report.[1]

Step-by-step pipeline

Here is the full pipeline to actually build the platform as a hackathon prototype.

Phase 0: Finalize use case

Pick one procurement category, such as riot gear, construction works, or vehicles, because the report itself suggests testing with three tender types later.[1]
For the prototype, one domain is enough because it helps you keep criteria, document formats, and UI more coherent.[1]

Do this:

- Choose one sample tender.
- Create 3 to 5 mock bidders.
- Give each bidder a realistic mix of clean, noisy, and ambiguous documents.
- Define expected human verdicts for benchmarking.

Phase 1: Design data model

The report strongly centers MongoDB, with collections for tenders, bidders, criteria manifests, extracted evidence, evaluation events, routing logs, clarification requests, and LLM interactions.[1]
For the prototype, you can keep a smaller schema but preserve the same structure so your system still looks enterprise-ready.[1]

Suggested collections:

- `tenders`
- `criteria_manifest_versions`
- `bidders`
- `documents`
- `extracted_evidence`
- `evaluation_events`
- `clarification_requests`
- `audit_exports`

Most important object is the criterion-level event, because the report treats one criterion result for one bidder as the basic atom of audit.[1]

Phase 2: Build tender upload and manifest generation

Upload tender PDF and corrigenda, extract text, split into clauses, and convert key clauses into structured criteria.[1]
The final output should be a reviewable manifest showing criterion name, threshold, type, evidence needed, and whether it is hard or graduated.[1]

Prototype implementation:

1. Upload PDF.
2. Extract text with PyMuPDF.
3. Detect sections using headings and regex.
4. Identify lines with numbers, percentages, dates, “must,” “shall,” “required,” “minimum,” and certificate names.
5. Convert to JSON criteria list.
6. Let user manually edit and approve manifest.

This step is important because the manifest becomes the whole evaluation blueprint later.[1]

Phase 3: Build bidder document ingestion

Upload bidder files and process them page by page, just like the report’s routing concept.[1]
You do not need full Celery in the first demo, but background jobs make the product feel more real if you have time.[1]

Pipeline:

1. Convert Word/Excel to PDF if required.
2. Check if page has text layer.
3. If yes, use direct extraction.
4. If scan/image, run OCR.
5. Classify page type: financial, certificate, BoQ, experience letter, ID/compliance.
6. Store extracted page text and metadata.

Useful extracted fields:

- bidder_name
- PAN
- GSTIN
- certificate_name
- certificate_validity
- turnover_year_1 / 2 / 3
- local_content_declared
- boq_local_total

- boq_grand_total
- work_order_description
- work_completion_date

Phase 4: Build evidence extraction

Now create smaller extractors for each criterion type.[1]
This is where you turn document text into structured evidence JSON.[1]

Examples:

- Turnover extractor: find financial values for required years.
- GSTIN extractor: regex + checksum validation.
- PAN extractor: regex.
- ISO validity extractor: find certificate date and expiry.
- Experience extractor: detect project title, client, amount, date.
- BoQ extractor: parse table rows and calculate local percentage.

The report suggests confidence scoring using OCR confidence, layout confidence, and NER confidence together.[1]
For your prototype, a simpler version is enough: combine OCR confidence with rule match confidence and assign High / Medium / Low.[1]

Phase 5: Build evaluation engine

This is the main logic layer.[1]
The report specifically recommends deterministic, rule-based evaluation with a three-state verdict engine and versioned policy configuration.[1]

Your prototype evaluation engine should:

- Read one criterion from manifest.
- Read matching evidence.
- Apply rule from config.
- Return:
  - extracted value
  - threshold
  - evidence source
  - confidence
  - auto_verdict
  - final_verdict
  - reason_code

Example:

- Criterion: turnover >= 50 Cr.[1]
- Extracted average: 51.7 Cr.[1]
- Confidence: 0.92.[1]
- Verdict: Eligible.[1]

Another example:

- Criterion: similar works in last 7 years.[1]
- Evidence found but wording is subjective.[1]
- Verdict: Needs Manual Review.[1]

This “manual review on ambiguity” is one of the most important concepts in the report, and you should highlight it in your demo.[1]

Phase 6: Build audit trail

The report’s strongest differentiator is the immutable audit trail with append-only event records and SHA-256 chaining.[1]
Even if you cannot build full cryptographic chaining, you should definitely store one event per criterion verdict and show event history in UI.[1]

Minimum event fields:

- tender_id
- bidder_id
- criterion_id
- criterion_text
- evidence_doc
- page_no
- extracted_value
- threshold
- match_logic
- auto_verdict
- final_verdict
- reviewer_action
- timestamp
- hash / previous_hash

This feature will make your prototype look far more serious than a normal OCR app.[1]

Phase 7: Build review dashboard

The report describes a criterion-focused workbench, and this is exactly what judges will like seeing.[1]
So create one main dashboard with bidder-wise and criterion-wise views.[1]

Best UI sections:

- Left panel: criteria list with status badges.
- Middle panel: document viewer or page preview.
- Right panel: extracted values, threshold, logic, confidence, verdict.
- Bottom or modal: reviewer comment, override, reason code.

Important statuses:

- Eligible
- Not Eligible
- Needs Manual Review
- Missing Evidence
- Low Confidence
- Red Flag

Phase 8: Build report export

The report mentions Excel summary, signed PDF report, and RTI-ready ZIP export generated from MongoDB aggregations.[1]
For hackathon, build one downloadable summary report, preferably PDF or Excel, containing criterion-wise evidence and verdict.[1]

Best output format:

- Bidder name
- Criterion
- Extracted value
- Required threshold
- Verdict
- Reason code
- Evidence page reference
- Officer note
- Timestamp

This directly demonstrates explainability.[1]

Technical stack

The report suggests a strong stack: MongoDB, Next.js, React, TypeScript, PyMuPDF, pdfplumber, PaddleOCR, TrOCR, LayoutLMv3, Redis, Celery, Docker, Keycloak, Vault, and Kubernetes.[1]
For hackathon, you should simplify this into a practical stack that still matches the report philosophy.[1]

Recommended prototype stack:

Layer	Best prototype choice
Frontend	Next.js or React with TypeScript. [1]
Backend API	FastAPI in Python.
Database	MongoDB. [1]
File storage	Local filesystem or GridFS if possible. [1]
OCR	PaddleOCR or Tesseract. [1]
PDF parsing	PyMuPDF, pdfplumber. [1]
Rules engine	Python service using YAML/JSON policy file. [1]
Queue	Celery + Redis, optional for demo. [1]
Table extraction	pdfplumber/Camelot. [1]
Image enhancement	OpenCV, optional CLAHE. [1]

If you want fastest hackathon build:

- React + FastAPI + MongoDB
- PyMuPDF + PaddleOCR
- Regex/rules instead of trained ML
- Static mock documents
- Good dashboard + explanation layer

That will be much more achievable than implementing every advanced model from the report. [1]

Small technical concepts

Here are the important technical terms in simpler language.

- **Deterministic system**: Same input gives same output every time.[1]
- **Rule engine**: A module that checks conditions using fixed logic instead of free-form AI.[1]
- **Manifest**: Structured list of all evaluation criteria for a tender.[1]
- **Versioned registry**: Keeping each updated version instead of replacing old data.[1]

- **Append-only event store**: New decisions are added as new records; old ones are not overwritten.[1]
- **Hash chaining**: Each event stores a hash linked to the previous one, making tampering detectable.[1]
- **Change streams**: Real-time database updates for live UI refresh.[1]
- **GridFS**: MongoDB method for storing large files.[1]
- **Confidence score**: Reliability estimate for extracted data.[1]
- **Layout analysis**: Finding where text blocks, tables, and fields are located on a page.[1]
- **Entity resolution**: Confirming that two slightly different names refer to the same bidder.[1]
- **CRDT/Yjs**: Technology for real-time collaborative editing.[1]
- **Air-gapped deployment**: System runs in isolated government infrastructure for security.[1]

Best demo story

Your presentation should not say, "we built an OCR system." The report clearly positions the product as an accountability platform, not a generic document reader.[1]
So your demo flow should be:

1. Upload a tender.[1]
2. Show extracted criteria manifest.[1]
3. Upload bidder documents.[1]
4. Show extracted evidence with confidence.[1]
5. Run evaluation.[1]
6. Show one clear pass, one clear fail, and one ambiguous manual-review case.[1]
7. Override one result with a reason code.[1]
8. Export audit report.[1]

This exact flow proves the full value of the report better than trying to show too many half-built features.[1]

5-week plan

The report itself gives a 5-phase roadmap: tender understanding, document ingestion, evaluation engine, human review plus audit trails, and then integration/testing.[1]
You can directly map your hackathon work to that same structure.[1]

Week 1

- Finalize one tender domain.
- Create MongoDB schema.
- Build tender upload.
- Extract text and create criteria manifest UI.[1]

Week 2

- Build bidder document upload.
- Add PDF text extraction and OCR.
- Create page-type routing.
- Extract core fields like PAN, GSTIN, turnover, certificate validity.[1]

Week 3

- Build rules engine.
- Implement Eligible / Not Eligible / Manual Review state machine.
- Add indigenous content calculation from BoQ if possible.[1]

Week 4

- Build reviewer dashboard.
- Add override workflow and reason codes.
- Create evaluation event logging and audit timeline.[1]

Week 5

- Polish UI.
- Create report export.
- Test with 3 to 5 mock bidders.
- Record demo and prepare explanation slides.[1]

What to skip

The report includes advanced ideas that are excellent for a full platform but too heavy for a hackathon prototype.[1]
You should skip or mock these unless your team is very strong and has time.[1]

Skip first:

- Full Indic-BERT training.[1]
- Full TrOCR fine-tuning.[1]
- Full committee real-time collaboration.[1]
- Government blacklist APIs.[1]
- Kubernetes production deployment.[1]
- Full DSC signing and secure cloud setup.[1]

Mock if needed:

- Corrigendum dependency graph.[1]

- Hash chain verification screen.[1]
- Clarification request email flow.[1]

KPIs to claim

The report proposes KPIs such as criterion extraction F1 >= 0.92, digital key-field extraction F1 0.95, handwritten extraction F1 0.78, auto-resolution rate >= 60%, zero false rejections due to low confidence without review, officer time savings >= 50%, and full traceability of decisions.[1]

For your prototype, do not claim all numbers unless tested; instead, show measured mock results on your own sample dataset and align them with these KPI categories.[1]

Final build recommendation

The best version of this project for the hackathon is:

- A web platform where an officer uploads a tender and bidder documents.[1]
- The system extracts criteria into a structured manifest.[1]
- It extracts bidder evidence from PDFs/scans.[1]
- It runs deterministic eligibility checks.[1]
- It marks unclear cases as Manual Review instead of guessing.[1]
- It lets the officer override with reason codes.[1]
- It generates a criterion-wise audit report with document references.[1]

That would represent the report faithfully because the report's main message is that **trustworthy procurement AI is not about replacing officers; it is about making every decision transparent, consistent, and defensible.**[1]

Would you like me to turn this into a very practical module-by-module development blueprint next, with folder structure, database schema, API endpoints, and exact screens to build?

⁂

Now tell me step by step process to build it completely from basic to scratch ie from frontend to bacend to db to ml to llm to api and so on.....

tell me the step by step entire process like what i have to do and how i can do it

You can build this system in clear phases: first design your data model and backend APIs, then implement tender and bidder pipelines, then the rule-based evaluation and audit trail, and finally add ML/LLM enhancements and a polished officer UI.[2]
Below is a concrete, end-to-end roadmap from zero to full prototype, including what to do and how to do it at each layer.

High-level build strategy

You want a working prototype that implements the four subsystems from the report:

- Tender Understanding Engine (TUE).
- Bidder Document Intelligence (BDI).
- Evaluation & Ambiguity-Resolution Engine (EARE).
- Human-in-the-Loop Workbench & Audit System (HLWAS).[2]

All of this sits on MongoDB, exposed via backend APIs, with a React/Next.js frontend, plus Python ML/OCR services for extraction.[2]

Recommended tech stack for you:

- Frontend: Next.js (React + TypeScript).
- Backend: FastAPI (Python) for APIs and ML/OCR integration.
- Database: MongoDB (Atlas or local).
- ML/OCR: PyMuPDF + pdfplumber + PaddleOCR/Tesseract + simple classifiers.[2]
- Auth: Simple JWT or session auth for prototype (Keycloak is overkill at hackathon scale).[2]

Phase 0: Scope and project setup

1. **Narrow the use case**
   - Pick 1 tender domain (e.g., riot gear or construction) and create 1 real tender PDF + 3–5 mock bidders.[2]
   - Define 5–8 criteria: turnover, ISO validity, similar work, indigenous content %, GSTIN/PAN, experience in last 7 years.[2]
2. **Create repos and folder structure**
   Suggested mono-repo layout:
   - `/backend/` – FastAPI app.
   - `/ml-services/` – optional if you separate heavy OCR/ML.
   - `/frontend/` – Next.js app.
   - `/infra/` – Docker, docker-compose, configs.
3. **Initialize tools**
   - Setup Python env (poetry/venv) with FastAPI, uvicorn, pymongo/motor, pydantic, PyMuPDF, pdfplumber, Pillow, OpenCV, PaddleOCR or Tesseract.[2]
   - Setup Node env with Next.js, TypeScript, Zustand or Redux for state, and a UI library (MUI, Chakra, or Tailwind + Headless UI).[2]

Phase 1: Database & data model (MongoDB)

The report places MongoDB at the center as document database and event store.[2]
Design your schemas first so everything else fits nicely.

1. **Define core collections**
   At minimum:
   - `tenders` – tender metadata and document references.[2]
   - `criteria_manifest_versions` – versioned list of criteria per tender.[2]
   - `bidders` – bidder profile and IDs.[2]
   - `documents` or `extracted_evidence` – per-page extraction and field-level values.[2]
   - `evaluation_events` – one document per (tender, bidder, criterion) decision.[2]
   - `clarification_requests` – optional for POC.[2]
2. **Create simple schema models**
   Example `evaluation_events` (simplified):
   - `tender_id`, `bidder_id`, `criterion_id`.
   - `criterion_text`, `evidence_docs[]` (doc_id, page, bbox).[2]
   - `extracted_values[]` (field, raw, normalized, confidence).[2]
   - `match_logic` (string like `"avg(52.1, 48.6, 54.4) > 50"`).[2]
   - `auto_verdict` (Eligible, Not Eligible, Manual Review).[2]
   - `final_verdict` (after officer review).[2]
   - `uncertainty_scores`, `reviewer_actions[]`, timestamps.[2]
3. **Indexes**
   - On `tenders(tender_id);bidders(bidder_id);criteria_manifest_versions(tender_id, version);evaluation_events(tender_id, bidder_id, criterion_id, event_timestamp)` like in the report.[2]

Phase 2: Backend skeleton & core APIs

Start with a clean FastAPI app exposing core endpoints, even with dummy data.

1. **Set up FastAPI project**
   - `app/main.py` with root router.
   - Connect to MongoDB (e.g., with Motor).
   - Add pydantic models matching your schemas.
2. **Implement core routes (first pass)**
   - `POST /tenders` – upload tender PDF(s), store metadata and file path/GridFS ID.[2]
   - `GET /tenders/{tender_id}` – tender details.
   - `POST /tenders/{tender_id}/generate_manifest` – triggers TUE.[2]

- `GET /tenders/{tender_id}/criteria_manifest` – latest manifest.[2]
- `POST /tenders/{tender_id}/bidders` – register bidder, upload zip/PDFs.[2]
- `POST /evaluate/{tender_id}/{bidder_id}` – run full evaluation pipeline.[2]
- `GET /evaluation/{tender_id}/{bidder_id}` – aggregated verdicts.[2]

3. **Add simple auth**
   - Use FastAPI OAuth2PasswordBearer + JWT for login.
   - Store officers in `users` collection with roles (officer, admin).

Now you have a working API skeleton; next you fill the internal logic step by step.

Phase 3: Implement Tender Understanding Engine (TUE)

You now implement back-end logic for extracting criteria from tender PDFs.[2]

1. **PDF ingestion and text extraction**
   In `/backend/app/services/tender_ingestion.py`:
   - Use PyMuPDF or pdfplumber to extract text and rough section headings.[2]
   - Split by headings like `Section`, `Clause`, `Eligibility`, `Qualifying Requirements`, etc.[2]
2. **Clause segmentation**
   - Break text into sentences (spaCy or simple sentence splitter).
   - Attach metadata: section name, page number, position.
3. **Rule-based clause classification (Phase 1)**
   Instead of fine-tuned Indic-BERT, begin with heuristic rules:
   - If contains "turnover", "balance sheet", numbers + `Cr` → `financial_threshold`.[2]
   - If contains "experience", "similar works", years → `experience_requirement`.[2]
   - If contains "IS:", "BIS", "ISO" → `certification_requirement`.[2]
   - If contains "technical specification" or model names → `technical_spec`.[2]
   - Else mark `non_criterion` or `compliance_condition`.[2]

   Later you can swap this for a small fine-tuned classifier.
4. **Hard vs. graduated detection**
   Implement simple logic from the report:[2]
   - If numeric + comparator (≥, <=, greater than) and no subjective adjectives → `hard`.
   - If "similar", "comparable", "adequate", "satisfactory" → `graduated`.[2]
   - If presence-only like "must have valid ISO 9001" → `hard_presence`.[2]
5. **Normalization**
   - Convert money to crores.
   - Normalize dates to ISO 8601.
   - Normalize units.[2]

6. **Build manifest and store version**
   - Create `criteria_manifest_version` document with structured list of criteria.[2]
   - Save as version 1, `status="draft"`.
   - Expose API `GET /tenders/{id}/criteria_manifest` to show to frontend.[2]
7. **Officer approval**
   - Frontend allows officer to edit text/thresholds and then press "Approve".
   - On approval, set `status="approved"` and maybe increment version if changed.[2]

Now you have a TUE pipeline from tender upload to an approved manifest blueprint.[2]

Phase 4: Implement Bidder Document Intelligence (BDI)

Next build ingestion and extraction for bidder documents.[2]

1. **Bidder upload API**
   - `POST /tenders/{tender_id}/bidders/{bidder_id}/documents`.
   - Accept multiple PDFs or a ZIP file.
   - Store file metadata in `documents` collection and physical files on disk or GridFS.[2]
2. **Per-page routing**
   For each page:
   - Check if it has a text layer → if yes, treat as digital PDF.[2]
   - Else treat as scanned image and run OCR.
   - Optionally run a simple CNN or heuristics to label page type: certificate, financial statement, BoQ, letter.[2]
3. **Type-specific extraction**
   Implement Python functions:
   - `extract_financials(page_text)` – get turnover for 3 years using regex and context.[2]
   - `extract_certificate(page_text)` – get ISO/BIS names and validity dates.[2]
   - `extract_experience(page_text)` – get project name, client, amount, completion date.[2]
   - `extract_boq(table)` – parse table rows and compute local vs total amounts.[2]

   Use pdfplumber/Camelot for tables, and PaddleOCR or Tesseract for scanned tables.[2]
4. **Confidence estimation**
   Basic version:
   - Take OCR confidence from OCR engine where available.[2]
   - Add simple rules: if value appears only once, attach medium confidence; if multiple consistent occurrences, raise to high.
   - Store `confidence` as 0–1.
5. **Store structured evidence**

Write to `extracted_evidence` collection:

- Key: `(tender_id, bidder_id)`.
- For each document: list of extracted fields (name, value, normalized_value, confidence, source_doc, page).[2]

This gives EARE a clean input to work on.[2]

Phase 5: Implement Evaluation & Ambiguity-Resolution Engine (EARE)

Now build the rule engine and three-state verdict machine.[2]

1. **Policy config**
   Create `policy.yaml` or JSON:
   - `financial_tolerance: 0.05` (5%).[2]
   - `ocr_confidence_threshold: 0.75`.[2]
   - Rules for each criterion type: how to compare, what to do with missing/low confidence.[2]
2. **Evidence-to-criterion mapping**
   For each criterion in manifest:
   - Use `evidence_template` (e.g., needs `turnover` field from `financial_statement` doc).[2]
   - Query `extracted_evidence` for fields matching that template.[2]
3. **Three-state verdict engine**
   Implement the exact table from the report:[2]
   - If evidence quality = HIGH, criterion clarity = HARD, match PASS → `Eligible`.
   - If evidence quality = HIGH, HARD, match FAIL → `Not Eligible`.
   - If HIGH, HARD, match ambiguous OR HIGH, GRADUATED → `Needs Manual Review`.
   - If evidence LOW or missing → `Needs Manual Review`.[2]

   And financial tolerance: within 5% is PASS, outside is FAIL.[2]
4. **Generate match logic strings**
   For each criterion, compute and store explanation strings, such as:
   - `"avg(52.1, 48.6, 54.4) = 51.7 > 50 (tolerance 5%)"`.[2]
   - `"Computed local content 58% < required 50% but declaration says 65% → DeclarationMismatch"`.[2]
5. **Write evaluation_events**
   For each criterion:
   - Construct event document with evidence docs, extracted values, match logic, auto verdict, uncertainty scores.[2]
   - Insert as a new document (append-only).[2]

This step gives your prototype real intelligence and is the core of eligibility analysis.[2]

Phase 6: Implement explainability & audit trail

Now you add the "government-grade" features that make this stand out.[2]

1. **Append-only event sourcing**
   - Ensure your code never updates/deletes from `evaluation_events`; always insert new events when something changes.[2]
   - Add `prior_event_id` to link back to the earlier event for that criterion.[2]
2. **SHA-256 chaining (optional but impressive)**
   - On every event insert, compute `event_hash = SHA256(prior_event_hash + json_payload)`.[2]
   - Store `event_hash` and `prior_event_hash`.
   - Provide an admin endpoint to recompute chain and verify integrity.
3. **Natural language explanations**
   - Implement a template:
     `"Bidder {name} is {final_verdict} for criterion {criterion_id} because {match_logic}. Evidence from {doc_name}, page {page}."` [2]
   - Use only stored event data; no magic.[2]
4. **Audit export**
   - Implement a report generator that aggregates `evaluation_events` per bidder, grouped by criterion.[2]
   - Output as:
     - HTML page that can be printed as PDF, or
     - XLSX using openpyxl.[2]

The export should show: tender ID, bidder, criterion, extracted value vs threshold, verdict, reason code, evidence doc page, officer, timestamp.[2]

Phase 7: Build the Human-in-the-Loop web frontend

Now you expose everything with a clean officer UI matching the HLWAS ideas.[2]

1. **Authentication and layout**
   - Login page for officer.
   - Main layout with sidebar (tenders, bidders) and content area.
2. **Tender manifest review UI**
   - Screen: list of extracted criteria (table with text, type, threshold, hard/graduated).[2]
   - Allow inline edit and "Approve Manifest" button.[2]
3. **Bidder evaluation UI (criterion-focused workbench)**
   Emulate the tri-pane layout described:

- Top panel: criterion text, threshold, type, auto verdict.[2]
- Middle panel: document viewer (PDF rendered with OpenSeadragon or PDF.js) plus overlays for evidence box and text.[2]
- Bottom panel: extracted values, match logic, confidence bars, verdict dropdown, reason code select, comment box.[2]

4. **Override workflow**
   - When officer changes verdict, require selecting a reason code (`TOLERANCE_APPLIED`, `SIMILAR_WORK_PRECEDENT`, `OTHER`).[2]
   - If `OTHER`, force entering free-text note.[2]
   - Send override as POST `/evaluation/{tender_id}/{bidder_id}/{criterion_id}/override`.
   - Backend writes new `evaluation_event` with reviewer action.[2]
5. **Dashboard view**
   - Summary cards: number of criteria auto-resolved vs manual review required.[2]
   - Table: rows per bidder with counts of Eligible/Not Eligible/Needs Review.[2]
6. **Audit view**
   - For a given bidder, show all events as a timeline: version history per criterion with timestamps and officers.[2]

This front-end is what sells your solution to judges because it showcases explainability and human control.[2]

Phase 8: Add ML components properly

Once rule-based prototype works, you can add ML as in the report, but with bounded roles.[2]

1. **Clause classifier**
   - Collect ~few hundred tender clauses labelled into financial, experience, technical, certification, compliance, non-criterion.[2]
   - Fine-tune a small Transformer (e.g., IndicBERT or any multilingual BERT) for classification.[2]
   - Deploy as a Python service called by TUE to auto-label clauses, but still allow officer correction.[2]
2. **Page type classifier**
   - Train a lightweight CNN or use LayoutLMv3 to detect page type: certificate, financial statement, BoQ, letter.[2]
   - Use this to route pages to correct extraction logic.[2]
3. **Improved OCR pipeline**
   - Integrate PaddleOCR for multi-lingual text.[2]

- Use CLAHE or Real-ESRGAN for low-quality scans.[2]
- Implement dual-pass OCR: if confidence < threshold, try enhanced preprocessing and mark field UNRELIABLE if still low.[2]

4. **Entity resolution**
   - Use fuzzy matching (Levenshtein) for bidder names anchored on PAN/GSTIN.[2]
   - Escalate to manual review if IDs don't align.[2]

At every point, ensure ML only assists extraction and classification; it never directly sets final verdicts.[2]

Phase 9: LLM integration (strictly advisory)

The report is explicit: LLMs are used only for suggestions and summaries, not for final eligibility decisions.[2]

You can integrate an LLM in two safe ways:

1. **Clause suggestion**
   - After rule-based extraction, call an LLM with tender text to suggest additional potential criteria or highlight ambiguous wording for officer review.[2]
2. **Evidence summarization**
   - Given multiple documents for a criterion, ask LLM to summarize relevant evidence, but still feed the structured numeric values from rule engine into evaluation events.[2]

Log all LLM interactions in `llm_interactions` with prompt, response, timestamp, and model version.[2]

Phase 10: Integration, testing, and demo flow

The report itself proposes a 5-phase implementation and success metrics; you can align your testing story with that.[2]

1. **End-to-end test on 3–5 mock bidders**
   - Run full pipeline: tender upload → manifest → bidder ingestion → evaluation → officer overrides → audit export.[2]
   - Check that all criteria have a decision and every decision is traceable.
2. **Measure basic KPIs**
   - Auto-resolution rate: % criteria resolved without manual review.[2]
   - Number of "Needs Manual Review" due to low confidence vs genuine ambiguity.[2]
   - Time taken vs manual process (estimate in demo).
3. **Prepare demo script**
   Live flow:

- Upload tender.
- Show generated manifest and officer approval.[2]
- Upload one bidder's docs.[2]
- Show background processing and evidence extraction.[2]
- Run evaluation: highlight one clear Eligible, one clear Not Eligible, one Needs Manual Review.[2]
- As officer, open manual review case, inspect evidence, override with reason.[2]
- Finally, export audit report and show how you can justify every decision.[2]

This end-to-end story matches exactly what the original idea promises and proves you built the system "from scratch" across frontend, backend, DB, ML, and LLM.[2]

If you want, next I can give you a concrete technical blueprint: exact MongoDB schemas, FastAPI endpoint definitions, and the main React pages/components you should implement in order.

⁂

yes give me every thing which is needed to build this

so that i can selected into the top 50 teams

To get into the top 50, you need three things:

1. A **coherent, working vertical slice** of PARAKH (end-to-end, not just ML snippets).
2. A **clear architecture + code quality** that matches the depth of your report.
3. A **story/demo** that screams "we understand CRPF procurement and accountability." [3]

Below is a concrete blueprint of **what to build** and **how**, layer by layer: DB, backend, ML/OCR, LLM, frontend, infra, and hackathon strategy.

1. Final architecture & tech choices

From your report, the target architecture is:[3]

- **Services:**
  - TUE (Tender Understanding Engine)
  - BDI (Bidder Document Intelligence)
  - EARE (Evaluation & Ambiguity Resolution Engine)
  - HLWAS (Human-in-the-Loop Workbench & Audit System) [3]
- **Backbone**: MongoDB as main DB + event store.[3]
- **Async processing**: Celery + Redis for heavy document/OCR tasks.[3]

- **Frontend:** Next.js, TypeScript, Yjs for committee mode later.[3]

Prototype-friendly tech stack (recommendation):

- **Frontend:**
  - Next.js + TypeScript + Tailwind or Chakra UI.
  - OpenSeadragon or PDF.js for document viewer.[3]
- **Backend API:**
  - FastAPI (Python) – easy integration with ML/OCR.
  - Motor/pymongo for MongoDB.
- **ML/OCR:**
  - PyMuPDF, pdfplumber, Camelot for text/tables.[3]
  - PaddleOCR for OCR; Tesseract as fallback.[3]
  - Simple BERT-based clause classifier if time.[3]
- **Infra:**
  - Docker for backend + frontend.
  - Single docker-compose cluster with MongoDB + Redis.

This will let you demo a serious system but still finish in time.

2. MongoDB data model (core schemas)

These are the **minimum** collections you should implement, aligned with your report.[3]

2.1 tenders

Stores tender metadata and linked documents.[3]

- `tender_id` (string, unique)
- `title`, `department`, `issue_date`, `bid_submission_date`
- `documents[]`:
  - `doc_id`, `type` (NIT, QR, Annexure, Corrigendum, Pre-bid Minutes), `gridfs_id`, `uploaded_at`
- `status` (DRAFT, ACTIVE, CLOSED)

2.2 criteria_manifest_versions

Versioned manifest is the blueprint for evaluation.[3]

- `manifest_id` (UUID)
- `tender_id`
- `version` (int)

- effective_date
- criteria[] (embedded):
  - criterion_id (e.g., ELIG-FIN-003)
  - short_name (e.g., "Average Turnover >= 50 Cr")
  - full_text (original clause)
  - type (financial_threshold, experience_requirement, technical_spec, certification_requirement, compliance_condition) [3]
  - classification (hard, graduated, hard_presence) [3]
  - normalized_threshold (e.g., 50.0 Cr) [3]
  - mandatory (bool)
  - evidence_template (expected fields and allowed doc types) [3]
- supersedes_manifest_id
- created_by, approved_by, status (DRAFT, APPROVED) [3]

2.3 bidders

Bidder master.[3]

- bidder_id (string, PAN-based)
- tender_id
- name, gstin, pan, contact
- registration_status

2.4 documents / extracted_evidence

You can either:

- Store per-document metadata in documents and evidence in extracted_evidence, or
- Store all in extracted_evidence as an array grouped by (tender_id, bidder_id).[3]

Simpler for hackathon:

extracted_evidence:

- tender_id, bidder_id
- extraction_metadata (model versions, timestamp) [3]
- documents[]:
  - doc_id, type, filename, pages[]
    - page_no, page_image_gridfs_id, raw_text, fields[]:
      - field_name (e.g., turnover_fy_2023)
      - raw_value, normalized_value, confidence, source_bbox

2.5 evaluation_events

Atom of audit for each (tender, bidder, criterion).[3]

- `event_id` (UUID)
- `tender_id`, `bidder_id`, `criterion_id`
- `criterion_text`, `criterion_version`[3]
- `evidence_docs[]`:
  - `doc_name`, `page`, `bbox`, `doc_hash`, `page_image_gridfs_id`[3]
- `extracted_values[]`:
  - `field`, `raw`, `normalized`, `confidence`[3]
- `match_logic` (string explanation)[3]
- `auto_verdict` (Eligible, Not Eligible, Needs Manual Review)[3]
- `final_verdict`
- `uncertainty_scores` (readability, extraction, clarity)[3]
- `reviewer_actions[]` (officer id, action, reason_code, note, timestamp)[3]
- `model_versions` (rules, ocr, classifier)[3]
- `event_timestamp`
- `prior_event_id`, `event_hash`

Indexes: `(tender_id, bidder_id, criterion_id, event_timestamp desc)`.[3]

2.6 clarification_requests (optional POC)

- `request_id`, `tender_id`, `bidder_id`, `criterion_id`
- `created_by`, `status`, `upload_link`, `expires_at`[3]

3. Backend API design (FastAPI)

Here is a **concrete endpoint list** you should implement.

3.1 Auth & users

- `POST /auth/login` – issue JWT.
- `GET /users/me` – current officer profile.

3.2 Tender & manifest

- `POST /tenders`
  - Body: metadata + PDFs (NIT, QR, annexures, corrigenda).
  - Logic: save files, insert tender record.
- `POST /tenders/{tender_id}/generate-manifest`

- Logic: run TUE (PDF → text → clauses → classify → build criteria[]).[3]
- Returns draft manifest.

- `GET /tenders/{tender_id}/manifests/latest`
  - Returns latest manifest.
- `PUT /tenders/{tender_id}/manifests/{manifest_id}/approve`
  - Officer reviews in UI, then approves; sets `status="APPROVED"`.[3]

3.3 Bidders & documents

- `POST /tenders/{tender_id}/bidders`
  - Register bidder.
- `POST /tenders/{tender_id}/bidders/{bidder_id}/documents`
  - Upload bidder docs (PDFs/ZIP).
- `POST /tenders/{tender_id}/bidders/{bidder_id}/ingest`
  - Logic: BDI pipeline → run OCR, routing, extraction → fill `extracted_evidence`.[3]

3.4 Evaluation

- `POST /evaluate/{tender_id}/{bidder_id}`
  - Logic: For each criterion in APPROVED manifest:[3]
    - Find matching evidence.
    - Apply rules/policy.
    - Insert `evaluation_events`.[3]
- `GET /evaluation/{tender_id}/{bidder_id}`
  - Returns aggregated results (by criterion) for UI.
- `POST /evaluation/{tender_id}/{bidder_id}/{criterion_id}/override`
  - Body: `final_verdict`, `reason_code`, `note`.[3]
  - Logic: create new `evaluation_event` with reviewer action.

3.5 Audit & export

- `GET /audit/{tender_id}/{bidder_id}`
  - Returns full event history per criterion (for timeline UI).[3]
- `GET /audit/{tender_id}/{bidder_id}/export`
  - Generates PDF/XLSX summary and returns link.[3]

4. ML/OCR pipeline details (BDI & TUE)

4.1 TUE clause extraction & classification

Step 1: text extraction

- Use PyMuPDF to split by page and extract plain text.[3]

Step 2: clause segmentation

- Use simple heuristics: split on line breaks + periods + bullet markers.
- Attach (page_no, section_name) metadata.

Step 3: classification (basic → advanced)

- Basic: keyword regex rules to classify into types (financial, experience, etc.).[3]
- Advanced: fine-tune a small IndicBERT on labeled procurement clauses (~1,200 as per report) and run as a microservice.[3]

Step 4: hard vs graduated

- Check clause text for comparators (≥, >, <=, "not less than", numeric values) and subjective words ("similar", "adequate", "satisfactory").[3]
- Tag with `hard`, `graduated`, or `hard_presence`.[3]

4.2 BDI document routing

For each page of each bidder document:[3]

1. If text layer coverage > 60% → use digital path (pdfplumber).[3]
2. Else treat as image, run:
   - Photo detection (simple CNN or heuristics).[3]
   - OCR via PaddleOCR.[3]
3. Classify page type (certificate, financial_statement, BoQ_table, experience_letter) using LayoutLMv3 or simpler heuristics.[3]
4. Log routing in `document_routing_log` (optional).[3]

4.3 Field extraction examples

Financials:

- Use regex like `(\d[\d,\.]*\s*(Cr|crore|lakhs|lakh))` and year context. [3]
- Normalize to crores (if "5,210 lakhs" → 52.1 Cr).[3]

Certificates:

- Search for "ISO", "BIS", "IS:" patterns.[3]
- Extract certificate number and date strings, parse with `dateparser`.[3]

Experience:

- Look for phrases "`work order`", "`completion certificate`", "`value of work`", "`client`".[3]
- Extract project description, amount, completion date.

BoQ (indigenous content):

- Detect tables via pdfplumber or Camelot.[3]
- Identify columns mapping to `local` and `total` amounts (normalize headers).[3]
- Sum `local_total` and `grand_total` and compute `computed_local_pct`.[3]

4.4 Confidence estimation

Use simplified composite:

- If OCR engine gives character-level confidence, average for region.[3]
- Combine with regex certainty and presence in multiple sources:
  - `confidence = 0.5*ocr_conf + 0.3*format_score + 0.2*consistency_score`.[3]
- If < 0.75, mark field as "LOW_CONFIDENCE" and escalate criterion to manual review.[3]

5. Rule engine design (EARE)

This is your **core differentiator**: deterministic 3-state state machine.[3]

5.1 Policy config

`policy.yaml` example:[3]

```
financial_tolerance: 0.05
ocr_confidence_threshold: 0.75
second_pass_enabled: true
ambiguity_rules:
  - name: missing_document
    condition: "required_doc not in uploaded"
    action: escalate_to_manual
```

5.2 Evaluation algorithm (per criterion)

For each `(tender, bidder, criterion)`:

1. Get criterion details from manifest.[3]
2. Use `evidence_template` to locate relevant fields in `extracted_evidence`.[3]
3. If missing or confidence < threshold → `Needs Manual Review`.[3]
4. If `classification = graduated` → `Needs Manual Review` (always).[3]
5. Else (hard):

- Compare normalized_value with threshold, using tolerance for financials.[3]
- PASS → `Eligible`.
- FAIL → `Not Eligible`.

6. Build `match_logic` string explaining the calculation.[3]
7. Insert `evaluation_event`.

Your verdict table should match the report exactly:[3]

Evidence Quality	Criterion Clarity	Match Strength	Auto-Verdict
HIGH	HARD	PASS	Eligible
HIGH	HARD	FAIL	Not Eligible
HIGH	HARD	AMBIGUOUS	Needs Manual Review
HIGH	GRADUATED	–	Needs Manual Review
LOW/MISSING	any	any	Needs Manual Review

6. Frontend: screens you must have

For top-50, your UI should clearly show "officer in control, AI assisting."[3]

6.1 Officer login & home

- Simple login.
- Dashboard cards:
  - Active tenders count.
  - Bidders evaluated.
  - Criteria auto-resolved vs manual.[3]

6.2 Tender manifest screen

- Upload tender docs.
- Button: "Generate Criteria Manifest".[3]
- Table of criteria:
  - Columns: ID, type, classification (hard/graduated), threshold, mandatory, ambiguity_flags.[3]
- "Approve Manifest" button after review.[3]

6.3 Bidder list & status

- View by tender:
  - Bidders in rows; columns: Eligible criteria, Not Eligible, Needs Review.[3]

6.4 Criterion-focused workbench (HLWAS)

Tri-pane:[3]

- **Top pane (criterion context):**
  - Clause text, type, threshold, auto verdict and final verdict, mandatory or desirable.[3]
- **Middle pane (evidence viewer):**
  - Document viewer (PDF.js/OpenSeadragon).[3]
  - Highlighted evidence region; displayed OCR text; confidence heatmap maybe as a colored overlay.[3]
- **Bottom pane (decision tools):**
  - Extracted_values table.
  - Match logic explanation.[3]
  - Auto verdict suggestion.
  - Dropdown: `Eligible / Not Eligible / Needs Manual Review`.
  - Reason code dropdown + comment input.[3]
  - Save button triggers `override` API.[3]

6.5 Audit trail & export

- Timeline view of all events for one bidder and criterion: auto evaluation, manual override, clarification uploads.[3]
- Button: "Download Audit Report" which calls backend export and downloads PDF/Excel.[3]

If you can also stub **Committee mode** (a view where multiple officers see same screen and votes appear), that will map directly to your report section 9.4, even if actual Yjs collaboration is minimal.[3]

7. Infra, security & deployment essentials

From the report's security and deployment section, you don't need full NIC cloud/K8s, but you should show awareness.[3]

Implement for hackathon:

- **Dockerized services:**
  - `backend` (FastAPI)

    - `frontend` (Next.js)
    - `mongo`
    - `redis` (for async queue).
- **Environment variables** for secrets (DB URIs, JWT keys).[3]
- **HTTPS** if you host externally (NGINX reverse proxy).
- **Simple RBAC** in code (roles: OFFICER, ADMIN), leaving Keycloak for future.[3]

Mention in your pitch that in a production deployment you'll:

- Host inside government cloud;
- Use Keycloak for IAM and RBAC;
- Use HashiCorp Vault for secrets;
- Enable at-rest encryption (MongoDB WiredTiger) and TLS.[3]

8. Hackathon strategy: what judges care about

To be top-50, you must show:

1. **Deep domain understanding** (already in report).[3]
2. **A real, working prototype** with end-to-end flow.
3. **Differentiation** vs "LLM over PDFs" teams.[3]

So focus on:

- The **three-state deterministic engine** and "Needs Manual Review" philosophy.[3]
- The **immutable, criterion-level audit trail** with event history and hashes.[3]
- Handling **messy, real-world documents** (mix of digital and scanned docs).[3]
- Officer-centric design (override, reason codes, clarification requests).[3]

9. Execution plan & team roles

Week 1–2: foundation

- One member: MongoDB + FastAPI schemas and endpoints.
- One member: Tender ingestion + manifest generation.[3]
- One member: OCR pipeline + simple extraction.
- One member: Next.js scaffolding + auth + basic screens.

Deliverable:

- Upload tender → generate manifest → view in UI.[3]

Week 3: BDI and EARE

- Implement bidder upload and BDI extraction for 3–4 key fields.[3]
- Implement evaluation_engine with 3–5 criteria fully working.[3]
- Write evaluation_events with match_logic.[3]
- Frontend: bidder status screen + basic workbench.

Deliverable:

- Upload bidder documents → run evaluation → see verdicts in UI.[3]

Week 4: HLWAS and audit trail

- Implement override & reason codes.[3]
- Implement audit timeline and export.[3]
- Polish UI for tri-pane workbench.

Deliverable:

- Live demo of officer reviewing ambiguous cases and exporting audit-ready report.[3]

Week 5 (buffer): polishing & storytelling

- Add minimal ML classifier for clauses or page types.[3]
- Add simple LLM-backed explanation/summarisation (non-critical).[3]
- Optimize flows, fix bugs, create demo video.

10. Final checklist (if you can tick these, you're in great shape)

- [ ] `tenders`, `criteria_manifest_versions`, `bidders`, `extracted_evidence`, `evaluation_events` collections with correct indexes.[3]
- [ ] Endpoints: tender upload, manifest generation & approval, bidder upload & ingest, evaluate, override, audit export.[3]
- [ ] TUE pipeline: PDF → clauses → criteria manifest with type & hard/graduated flags.[3]
- [ ] BDI pipeline: digital and scanned PDFs → OCR → extracted fields with confidence.[3]
- [ ] Evaluation engine: deterministic 3-state verdict with tolerance and ambiguity handling.[3]
- [ ] Evaluation events: append-only with prior_event_id and event_hash.[3]
- [ ] HLWAS UI: tri-pane screen, reason codes on override, clarification request stub.[3]
- [ ] Audit report: criterion-wise explanation, doc references, officer notes.[3]
- [ ] Clear demo script showing one pass, one fail, and one manual review case.[3]

If you want next, I can help you with **concrete FastAPI models and endpoint signatures** plus **Next.js page/component structure** so you can start coding directly.

yes give me all required codes and other required items

I'll give you a **full working skeleton**: repo structure, `docker-compose`, FastAPI backend (models + routes + services), policy config, and Next.js frontend skeleton. You can copy–paste and then fill in details. All of this is aligned with your report's design (MongoDB, TUE/BDI/EARE/HLWAS, policy.yaml, evaluation_events).[4]

1. Repo & directory structure

Use a mono-repo:

```
parakh/
  docker-compose.yml
  backend/
    app/
      main.py
      config.py
      db.py
      models/
        tender.py
        bidder.py
        manifest.py
        evidence.py
        evaluation.py
        user.py
      routers/
        auth.py
        tender.py
        bidder.py
        evaluation.py
        audit.py
      services/
        tue_service.py
        bdi_service.py
        evaluation_service.py
        audit_service.py
      ml/
        clause_classifier.py   # optional
        ocr_pipeline.py
        boq_extraction.py
      core/
        security.py
        policy_loader.py
  frontend/
    app/  (Next.js /app router)
      layout.tsx
      page.tsx
      tenders/
```

```
      bidders/
      evaluation/
```

MongoDB and Redis will run from `docker-compose`.[4]

2. docker-compose.yml (Mongo + Redis + backend + frontend)

Minimal version:

```
version: "3.8"

services:
  mongo:
    image: mongo:6
    container_name: parakh-mongo
    restart: unless-stopped
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db

  redis:
    image: redis:7
    container_name: parakh-redis
    restart: unless-stopped
    ports:
      - "6379:6379"

  backend:
    build: ./backend
    container_name: parakh-backend
    restart: unless-stopped
    env_file:
      - ./backend/.env
    ports:
      - "8000:8000"
    depends_on:
      - mongo
      - redis

  frontend:
    build: ./frontend
    container_name: parakh-frontend
    restart: unless-stopped
    ports:
      - "3000:3000"
    depends_on:
      - backend

volumes:
  mongo-data:
```

`.env` example:

```
MONGO_URI=mongodb://mongo:27017/parakh
JWT_SECRET=supersecret
JWT_ALGORITHM=HS256
POLICY_FILE=/app/policy.yaml
```

3. Backend: basic FastAPI setup

3.1 `backend/app/main.py`

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.routers import auth, tender, bidder, evaluation, audit
from app.db import connect_to_mongo, close_mongo_connection

app = FastAPI(title="PARAKH API", version="0.1.0")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # tighten later
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.on_event("startup")
async def startup_db():
    await connect_to_mongo()

@app.on_event("shutdown")
async def shutdown_db():
    await close_mongo_connection()

app.include_router(auth.router, prefix="/auth", tags=["auth"])
app.include_router(tender.router, prefix="/tenders", tags=["tenders"])
app.include_router(bidder.router, prefix="/tenders/{tender_id}/bidders", tags=["bi
app.include_router(evaluation.router, prefix="/evaluation", tags=["evaluation"])
app.include_router(audit.router, prefix="/audit", tags=["audit"])
```

3.2 `backend/app/db.py`

```
from motor.motor_asyncio import AsyncIOMotorClient
from bson import ObjectId
import os

MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/parakh")

client: AsyncIOMotorClient | None = None
db = None
```

```
async def connect_to_mongo():
    global client, db
    client = AsyncIOMotorClient(MONGO_URI)
    db = client.get_default_database()

async def close_mongo_connection():
    global client
    if client:
        client.close()

def get_collection(name: str):
    return db[name]
```

4. Backend models (Pydantic) – key data structures

4.1 Criteria manifest models: `models/manifest.py`

```
from pydantic import BaseModel, Field
from typing import List, Optional
from datetime import datetime

class EvidenceTemplate(BaseModel):
    required_fields: List[str]
    allowed_doc_types: List[str]

class Criterion(BaseModel):
    criterion_id: str
    short_name: str
    full_text: str
    type: str  # financial_threshold, experience_requirement, etc.
    classification: str  # hard, graduated, hard_presence
    normalized_threshold: Optional[float] = None
    mandatory: bool = True
    evidence_template: EvidenceTemplate
    ambiguity_flags: List[str] = []

class CriteriaManifest(BaseModel):
    manifest_id: str
    tender_id: str
    version: int
    effective_date: datetime
    criteria: List[Criterion]
    supersedes_manifest_id: Optional[str] = None
    created_by: str
    approved_by: Optional[str] = None
    status: str = "DRAFT"
```

4.2 Evaluation event model: `models/evaluation.py`

```
from pydantic import BaseModel
from typing import List, Optional, Dict
from datetime import datetime

class EvidenceDoc(BaseModel):
    doc_name: str
    page: int
    bbox: List[int]
    doc_hash: str
    page_image_gridfs_id: Optional[str] = None

class ExtractedValue(BaseModel):
    field: str
    raw: str
    normalized: str
    confidence: float

class ReviewerAction(BaseModel):
    officer_id: str
    action: str  # confirm_auto, override
    reason_code: Optional[str] = None
    note: Optional[str] = None
    timestamp: datetime

class EvaluationEvent(BaseModel):
    event_id: str
    tender_id: str
    bidder_id: str
    criterion_id: str
    criterion_text: str
    criterion_version: str
    evidence_docs: List[EvidenceDoc]
    extracted_values: List[ExtractedValue]
    match_logic: str
    auto_verdict: str
    final_verdict: str
    uncertainty_scores: Dict[str, float]
    reviewer_actions: List[ReviewerAction]
    model_versions: Dict[str, str]
    event_timestamp: datetime
    prior_event_id: Optional[str] = None
    event_hash: str
```

This mirrors your Appendix A schema.[4]

5. Policy loader & rule engine

5.1 `core/policy_loader.py`

```
import yaml
import os

_policy = None

def load_policy():
    global _policy
    if _policy is None:
        policy_path = os.getenv("POLICY_FILE", "policy.yaml")
        with open(policy_path, "r") as f:
            _policy = yaml.safe_load(f)
    return _policy
```

5.2 `policy.yaml` (from your Appendix F)[4]

```
financial_tolerance: 0.05
ocr_confidence_threshold: 0.75
second_pass_enabled: true

ambiguity_rules:
  - name: missing_document
    condition: "required_doc not in uploaded"
    action: escalate_to_manual
```

You can extend this with rules per criterion type.

5.3 Evaluation engine: `services/evaluation_service.py`

```
from datetime import datetime
import hashlib
import uuid
from typing import Dict, Any, List

from app.db import get_collection
from app.core.policy_loader import load_policy

policy = load_policy()

def compute_event_hash(prior_hash: str | None, payload: Dict[str, Any]) -> str:
    base = (prior_hash or "") + str(payload)
    return hashlib.sha256(base.encode("utf-8")).hexdigest()

async def evaluate_bidder_for_tender(tender_id: str, bidder_id: str):
    manifest_col = get_collection("criteria_manifest_versions")
    evidence_col = get_collection("extracted_evidence")
    events_col = get_collection("evaluation_events")
```

```
    manifest = await manifest_col.find_one(
        {"tender_id": tender_id, "status": "APPROVED"},
        sort=[("version", -1)]
    )
    if not manifest:
        raise ValueError("No approved manifest")

    evidence = await evidence_col.find_one({"tender_id": tender_id, "bidder_id": b
    if not evidence:
        raise ValueError("No extracted evidence")

    prior_hash = None
    now = datetime.utcnow()
    results = []

    for c in manifest["criteria"]:
        criterion_id = c["criterion_id"]
        classification = c["classification"]
        ctype = c["type"]
        threshold = c.get("normalized_threshold")
        evid_tpl = c["evidence_template"]

        # 1) Map evidence to criterion
        ev_docs, extracted_values, evidence_quality = map_evidence_to_criterion(
            evidence, evid_tpl
        )

        # 2) Determine auto verdict using 3-state machine
        auto_verdict, match_logic, uncertainty_scores = apply_rules(
            c, extracted_values, evidence_quality, threshold
        )

        event_payload = {
            "event_id": str(uuid.uuid4()),
            "tender_id": tender_id,
            "bidder_id": bidder_id,
            "criterion_id": criterion_id,
            "criterion_text": c["full_text"],
            "criterion_version": f"v{manifest['version']}",
            "evidence_docs": ev_docs,
            "extracted_values": extracted_values,
            "match_logic": match_logic,
            "auto_verdict": auto_verdict,
            "final_verdict": auto_verdict,
            "uncertainty_scores": uncertainty_scores,
            "reviewer_actions": [],
            "model_versions": {"rules": "v1.0", "ocr": "v0.1"},
            "event_timestamp": now,
            "prior_event_id": None,  # fill when chaining
        }
        event_hash = compute_event_hash(prior_hash, event_payload)
        event_payload["event_hash"] = event_hash

        await events_col.insert_one(event_payload)
        prior_hash = event_hash
        results.append(event_payload)
```

```
    return results

def map_evidence_to_criterion(evidence_doc: Dict[str, Any], template: Dict[str, Any
    # Very simplified; you'll map by doc type & field names
    required_fields = template["required_fields"]
    docs = []
    values = []
    quality = "LOW"

    for doc in evidence_doc.get("documents", []):
        for page in doc.get("pages", []):
            for field in page.get("fields", []):
                if field["field_name"] in required_fields:
                    docs.append({
                        "doc_name": doc["filename"],
                        "page": page["page_no"],
                        "bbox": field.get("bbox", [0,0,0,0]),
                        "doc_hash": doc.get("hash", ""),
                        "page_image_gridfs_id": page.get("page_image_gridfs_id"),
                    })
                    values.append({
                        "field": field["field_name"],
                        "raw": field["raw_value"],
                        "normalized": field["normalized_value"],
                        "confidence": field["confidence"],
                    })
    if values and min(v["confidence"] for v in values) >= policy["ocr_confidence_thr
        quality = "HIGH"
    return docs, values, quality

def apply_rules(criterion: Dict[str, Any],
                values: List[Dict[str, Any]],
                evidence_quality: str,
                threshold: float | None):
    classification = criterion["classification"]
    ctype = criterion["type"]
    tolerance = policy.get("financial_tolerance", 0.05)
    uncertainty = {"readability": 0.9, "extraction": 0.9, "clarity": 0.9}
    if not values or evidence_quality != "HIGH":
        return "Needs Manual Review", "Insufficient or low-confidence evidence", unc

    if classification == "graduated":
        return "Needs Manual Review", "Graduated criterion requires officer judgmer

    # hard criteria
    if ctype == "financial_threshold" and threshold is not None:
        nums = [float(v["normalized"]) for v in values]
        avg_val = sum(nums) / len(nums)
        within_tol = avg_val + (avg_val * tolerance) >= threshold
        if within_tol:
            logic = f"avg({', '.join(str(n) for n in nums)}) = {avg_val:.2f} >= {thres
            return "Eligible", logic, uncertainty
        else:
            logic = f"avg({', '.join(str(n) for n in nums)}) = {avg_val:.2f} < {thresh
            return "Not Eligible", logic, uncertainty
```

```
    # Add more types (certification, presence-only, etc.)
    return "Needs Manual Review", "Rule not implemented yet", uncertainty
```

This implements your deterministic 3-state logic and hash chaining idea at a basic level.[4]

6. TUE and BDI skeletons

6.1 TUE: `services/tue_service.py`

```
import fitz  # PyMuPDF
from app.db import get_collection
from datetime import datetime
import uuid
from typing import List, Dict, Any

def extract_clauses_from_pdf(pdf_path: str) -> List[Dict[str, Any]]:
    doc = fitz.open(pdf_path)
    clauses = []
    for page_no, page in enumerate(doc, start=1):
        text = page.get_text()
        lines = text.split("\n")
        for line in lines:
            cleaned = line.strip()
            if len(cleaned) < 20:
                continue
            clauses.append({
                "page_no": page_no,
                "text": cleaned,
            })
    return clauses

def classify_clause(text: str) -> Dict[str, str]:
    lower = text.lower()
    # very simple starter
    if "turnover" in lower:
        return {"type": "financial_threshold", "classification": "hard"}
    if "similar work" in lower or "similar works" in lower:
        return {"type": "experience_requirement", "classification": "graduated"}
    if "iso" in lower or "bis" in lower:
        return {"type": "certification_requirement", "classification": "hard_presen
    # etc.
    return {"type": "non_criterion", "classification": "non_criterion"}

async def generate_manifest_for_tender(tender_id: str, pdf_path: str, officer_id: s
    manifest_col = get_collection("criteria_manifest_versions")
    clauses = extract_clauses_from_pdf(pdf_path)
    criteria = []
    idx = 1
    for cl in clauses:
        meta = classify_clause(cl["text"])
        if meta["type"] == "non_criterion":
            continue
```

```
        criterion_id = f"{tender_id}-C{idx:03d}"
        criteria.append({
            "criterion_id": criterion_id,
            "short_name": cl["text"][:80],
            "full_text": cl["text"],
            "type": meta["type"],
            "classification": meta["classification"],
            "normalized_threshold": None,
            "mandatory": True,
            "evidence_template": {
                "required_fields": [],
                "allowed_doc_types": [],
            },
            "ambiguity_flags": [],
        })
        idx += 1

    manifest = {
        "manifest_id": str(uuid.uuid4()),
        "tender_id": tender_id,
        "version": 1,
        "effective_date": datetime.utcnow(),
        "criteria": criteria,
        "supersedes_manifest_id": None,
        "created_by": officer_id,
        "approved_by": None,
        "status": "DRAFT",
    }
    await manifest_col.insert_one(manifest)
    return manifest
```

Later you can replace `classify_clause` with Indic-BERT.[4]

6.2 BDI OCR pipeline: `ml/ocr_pipeline.py`

```
import pytesseract
from pdf2image import convert_from_path
import cv2
import numpy as np
from typing import List, Dict, Any

def preprocess_image(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # simple CLAHE
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
    cl = clahe.apply(gray)
    return cl

def ocr_page_image(img) -> Dict[str, Any]:
    pre = preprocess_image(img)
    data = pytesseract.image_to_data(pre, output_type=pytesseract.Output.DICT)
    text = " ".join(data["text"])
    # You can compute confidence as mean conf of non -1
    confs = [c for c in data["conf"] if c != -1]
    avg_conf = sum(confs) / len(confs) / 100 if confs else 0.0
```

```
    return {"text": text, "confidence": avg_conf}

def pdf_to_ocr_results(pdf_path: str) -> List[Dict[str, Any]]:
    pages = convert_from_path(pdf_path, dpi=300)
    outputs = []
    for i, page in enumerate(pages, start=1):
        img = cv2.cvtColor(np.array(page), cv2.COLOR_RGB2BGR)
        result = ocr_page_image(img)
        outputs.append({
            "page_no": i,
            "text": result["text"],
            "ocr_confidence": result["confidence"],
        })
    return outputs
```

Then you can write extraction functions that parse `result["text"]` to find fields like turnover, ISO, etc.[4]

7. Routers (FastAPI endpoints) – examples

7.1 Tender router: `routers/tender.py`

```
from fastapi import APIRouter, UploadFile, File, Depends, HTTPException
from app.db import get_collection
from app.services.tue_service import generate_manifest_for_tender
import shutil
import uuid
import os

router = APIRouter()

UPLOAD_DIR = "/app/uploads/tenders"

@router.post("")
async def create_tender(title: str, department: str):
    tender_id = f"TEN-{uuid.uuid4().hex[:8].upper()}"
    tenders = get_collection("tenders")
    await tenders.insert_one({
        "tender_id": tender_id,
        "title": title,
        "department": department,
        "status": "DRAFT",
    })
    return {"tender_id": tender_id}

@router.post("/{tender_id}/upload")
async def upload_tender_pdf(tender_id: str, file: UploadFile = File(...)):
    os.makedirs(UPLOAD_DIR, exist_ok=True)
    file_path = os.path.join(UPLOAD_DIR, f"{tender_id}_{file.filename}")
    with open(file_path, "wb") as f:
        shutil.copyfileobj(file.file, f)
    # store ref in DB if needed
    return {"file_path": file_path}
```

```
@router.post("/{tender_id}/generate_manifest")
async def generate_manifest(tender_id: str, officer_id: str):
    # assume one main pdf path known; in real system, query DB
    pdf_path = f"/app/uploads/tenders/{tender_id}_NIT.pdf"
    manifest = await generate_manifest_for_tender(tender_id, pdf_path, officer_id)
    return manifest
```

7.2 Evaluation router: `routers/evaluation.py`

```
from fastapi import APIRouter, HTTPException
from app.services.evaluation_service import evaluate_bidder_for_tender
from app.db import get_collection

router = APIRouter()

@router.post("/{tender_id}/{bidder_id}")
async def run_evaluation(tender_id: str, bidder_id: str):
    try:
        results = await evaluate_bidder_for_tender(tender_id, bidder_id)
        return {"status": "ok", "events_created": len(results)}
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))

@router.get("/{tender_id}/{bidder_id}")
async def get_evaluation_summary(tender_id: str, bidder_id: str):
    events_col = get_collection("evaluation_events")
    cursor = events_col.find(
        {"tender_id": tender_id, "bidder_id": bidder_id}
    )
    events = [e async for e in cursor]
    # group by criterion_id for UI
    criteria_map = {}
    for e in events:
        cid = e["criterion_id"]
        if cid not in criteria_map:
            criteria_map[cid] = e
        else:
            # pick latest by timestamp
            if e["event_timestamp"] > criteria_map[cid]["event_timestamp"]:
                criteria_map[cid] = e
    return list(criteria_map.values())
```

8. Frontend: Next.js skeleton + key pages

Assume Next.js 13+ with `/app` router.

8.1 `frontend/app/layout.tsx`

```
import "./globals.css";
import React from "react";

export const metadata = {
  title: "PARAKH",
  description: "Tender evaluation & audit platform",
};

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
    <html lang="en">
      <body className="bg-slate-50 text-slate-900">
        <div className="min-h-screen flex">
          <aside className="w-64 bg-slate-900 text-slate-100 p-4">
            <h1 className="font-bold mb-4">PARAKH</h1>
            <nav className="space-y-2">
              <a href="/" className="block">Dashboard</a>
              <a href="/tenders" className="block">Tenders</a>
            </nav>
          </aside>
          <main className="flex-1 p-6">{children}</main>
        </div>
      </body>
    </html>
  );
}
```

8.2 Tender list page: `frontend/app/tenders/page.tsx`

```
"use client";

import React, { useEffect, useState } from "react";

interface Tender {
  tender_id: string;
  title: string;
  department: string;
  status: string;
}

export default function TendersPage() {
  const [tenders, setTenders] = useState<Tender[]>([]);

  useEffect(() => {
    fetch("http://localhost:8000/tenders")
      .then(r => r.json())
      .then(setTenders);
  }, []);

  return (
    <div>
      <h2 className="text-xl font-semibold mb-4">Tenders</h2>
```

```
      <table className="min-w-full text-sm">
        <thead>
          <tr>
            <th className="text-left p-2">Tender ID</th>
            <th className="text-left p-2">Title</th>
            <th className="text-left p-2">Department</th>
            <th className="text-left p-2">Status</th>
          </tr>
        </thead>
        <tbody>
          {tenders.map(t => (
            <tr key={t.tender_id} className="border-b">
              <td className="p-2">{t.tender_id}</td>
              <td className="p-2">
                <a href={`/tenders/${t.tender_id}`} className="text-blue-600 underlir
                  {t.title}
                </a>
              </td>
              <td className="p-2">{t.department}</td>
              <td className="p-2">{t.status}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

8.3 Criterion-focused workbench page (outline)

`frontend/app/evaluation/[tender_id]/[bidder_id]/page.tsx`

```
"use client";

import React, { useEffect, useState } from "react";

interface EvaluationEvent {
  criterion_id: string;
  criterion_text: string;
  match_logic: string;
  auto_verdict: string;
  final_verdict: string;
  extracted_values: { field: string; raw: string; normalized: string; confidence: n
  evidence_docs: { doc_name: string; page: number }[];
}

export default function EvaluationWorkbench({ params }: { params: { tender_id: strin
  const { tender_id, bidder_id } = params;
  const [events, setEvents] = useState<EvaluationEvent[]>([]);
  const [selected, setSelected] = useState<EvaluationEvent | null>(null);

  useEffect(() => {
    fetch(`http://localhost:8000/evaluation/${tender_id}/${bidder_id}`)
      .then(r => r.json())
      .then((data) => {
```

```
      setEvents(data);
      if (data.length > 0) setSelected(data[^4_0]);
    });
}, [tender_id, bidder_id]);

if (!selected) return <div>Loading...</div>;

return (
  <div className="grid grid-rows-[auto,1fr,auto] gap-4 h-[calc(100vh-5rem)]">
    {/* Top: criterion context */}
    <section className="border rounded p-3 bg-white shadow-sm">
      <h3 className="font-semibold mb-1">{selected.criterion_id}</h3>
      <p className="text-sm mb-2">{selected.criterion_text}</p>
      <div className="flex items-center gap-4 text-sm">
        <span>Auto verdict: <strong>{selected.auto_verdict}</strong></span>
        <span>Final verdict: <strong>{selected.final_verdict}</strong></span>
      </div>
    </section>

    {/* Middle: evidence + list */}
    <section className="grid grid-cols-[240px,1fr] gap-4">
      {/* Criterion list */}
      <div className="border rounded bg-white overflow-y-auto">
        {events.map(ev => (
          <button
            key={ev.criterion_id}
            onClick={() => setSelected(ev)}
            className={`w-full text-left px-3 py-2 text-sm border-b ${
              selected.criterion_id === ev.criterion_id ? "bg-blue-50" : ""
            }`}
          >
            <div className="font-medium">{ev.criterion_id}</div>
            <div className="truncate text-xs text-slate-500">{ev.criterion_text}</d
            <div className="text-xs">Verdict: {ev.final_verdict}</div>
          </button>
        ))}
      </div>

      {/* Evidence viewer placeholder */}
      <div className="border rounded bg-white p-3">
        <h4 className="font-semibold text-sm mb-2">Evidence</h4>
        <ul className="text-xs mb-3">
          {selected.evidence_docs.map((d, idx) => (
            <li key={idx}>{d.doc_name} - page {d.page}</li>
          ))}
        </ul>
        {/* Later: embed PDF viewer here */}
        <div className="h-64 border bg-slate-100 flex items-center justify-center
          PDF viewer placeholder
        </div>
      </div>
    </section>

    {/* Bottom: match logic, override controls */}
    <section className="border rounded p-3 bg-white">
      <h4 className="font-semibold text-sm mb-2">Extracted values</h4>
```

```
        <table className="text-xs mb-3">
          <thead>
            <tr>
              <th className="text-left pr-2">Field</th>
              <th className="text-left pr-2">Raw</th>
              <th className="text-left pr-2">Normalized</th>
              <th className="text-left pr-2">Conf</th>
            </tr>
          </thead>
          <tbody>
            {selected.extracted_values.map((v, idx) => (
              <tr key={idx}>
                <td className="pr-2">{v.field}</td>
                <td className="pr-2">{v.raw}</td>
                <td className="pr-2">{v.normalized}</td>
                <td className="pr-2">{(v.confidence*100).toFixed(1)}%</td>
              </tr>
            ))}
          </tbody>
        </table>

        <p className="text-xs mb-3">Logic: {selected.match_logic}</p>

        {/* Override form would trigger POST /evaluation/{tender}/{bidder}/{criteri
      </section>
    </div>
  );
}
```

This directly reflects your criterion-focused workbench idea.[4]

9. What next

With these skeletons you have:

- Mongo models aligned with the report.[4]
- FastAPI app with key routes (tenders, evaluation).
- Core evaluation engine with deterministic three-state logic.[4]
- TUE and OCR/BDI scaffolding functions.[4]
- Next.js pages for tenders list and evaluation workbench.

From here you should:

1. Fill in extraction functions for your chosen criteria (turnover, ISO, similar work, BoQ).[4]
2. Implement override API + frontend form.[4]
3. Add an audit export endpoint that aggregates `evaluation_events` into an Excel/PDF summary using openpyxl or a simple HTML-to-PDF.[4]

If you want, I can next help you sketch **concrete regex-based extractors** for turnover, ISO validity, PAN/GSTIN, and a BoQ parser, so you have ready-to-use ML-lite code for the key criteria.

⁂

1. Team-Zenith-Theme-3-AI-for-Bharat-Hackathon.pdf
2. Team-Zenith-Theme-3-AI-for-Bharat-Hackathon.pdf
3. Team-Zenith-Theme-3-AI-for-Bharat-Hackathon.pdf
4. Team-Zenith-Theme-3-AI-for-Bharat-Hackathon.pdf